

Compositionality of the ZAP Protocol Family: A Categorical Framework for Sub-Protocol Composition

ZAP Protocol Authors
zap-protocol.io

2026

Abstract

The ZAP protocol family ships as a layered specification: a wire format (Z_{base}) under several application protocols (`zap-http`, `zap-ws`, `zap-fix`, `zap-rns`, `zap-mcp`, `zap-acp`, `zap-a2a`). Specifications of this shape are usually composed informally, and the security guarantees of the parts are not always preserved when they are stacked. We give a categorical account of the family. We define a strict 2-category $\mathcal{C}_{\text{ZAP}}$ whose objects are protocols, whose 1-morphisms are session-preserving embeddings, and whose 2-morphisms are simulation refinements. We prove (i) $\mathcal{C}_{\text{ZAP}}$ satisfies the category and 2-category axioms, (ii) Z_{base} is initial in $\mathcal{C}_{\text{ZAP}}$, and (iii) every security property expressible as an indistinguishability game with non-negligible advantage is preserved along tight embeddings, with an explicit composition bound. The framework makes mechanical the engineering claim that, e.g., running `zap-fix-over-zap-rns` inherits the post-quantum and session-binding properties of both layers without re-proof. We discuss the relationship to Universal Composability [1], multiparty session types [2], and reactive simulatability [3].

1 Introduction

Protocol stacks are usually composed at design time and verified piecewise. The composition is left to a paragraph in the specification. This is fine when the composition is loose — e.g., HTTP-over-TCP, where the application layer treats the transport as an opaque byte stream. It is dangerous when the composition is tight — where the inner protocol relies on an invariant of the outer protocol that the outer protocol does not promise. The triple-handshake attack on TLS session resumption [4] is the canonical example: the inner channel binding broke because two TLS sessions were chained without a name binding.

The ZAP family is a layered specification with seven sub-protocols sitting over a common wire format. The temptation in such a setting is to write the spec in English and let implementers stitch the layers together. We resist that temptation. We give a categorical framework in which the composition is a primitive, security properties are parameterized over morphisms, and the engineer’s question “*if I run protocol P over protocol Q , what do I get?*” becomes a calculation rather than a discussion.

The contribution of this paper is structural, not cryptographic. We do not prove new lower bounds for any individual protocol’s security. We give a category in which the existing proofs about the parts can be *composed* mechanically into proofs about the whole, with explicit and tight bounds.

Roadmap. Section 2 recaps the seven sub-protocols. Section 3 defines the category $\mathcal{C}_{\text{ZAP}}$. Section 4 states and proves the central composition theorem (full proof in the companion document, with references inline). Section 5 works out the five concrete embeddings. Section 6 proves the universal

property of Z_{base} . Section 7 discusses what the framework buys us in practice. Section 8 situates the work.

2 The ZAP Sub-Protocol Family

We give a one-paragraph recap of each sub-protocol. Each is the subject of a separate paper in this collection; we cite those for the full specification and security claims.

Z_{base} (**zap-wire**). The base wire format. A length-prefixed, zero-copy frame carrying a typed envelope and a payload. Sessions are identified by 128-bit session IDs and are bound to the underlying transport via a key schedule derived from a hybrid post-quantum KEM [5]. Z_{base} is the substrate; every other protocol in the family ultimately produces Z_{base} frames on the wire.

Z_{http} (**zap-http**). A request/response protocol. Each request is a single Z_{base} frame; each response is a single Z_{base} frame. Idempotency is carried in the envelope. Z_{http} has no session-level state beyond the underlying Z_{base} session.

Z_{ws} (**zap-ws**). A bidirectional streaming protocol over a single Z_{base} session. Frames flow in both directions concurrently. Z_{ws} adds back-pressure annotations and stream multiplexing identifiers.

Z_{fix} (**zap-fix**). A FIX-compatible session protocol. Sequence numbers, gap-fill, persistent session identifiers. Z_{fix} carries FIX 4.4-style messages re-encoded as Z_{base} envelopes.

Z_{rns} (**zap-rns**). The identity-binding sub-protocol. Names are bound to public keys and to session keys at handshake time. Z_{rns} contributes the channel-binding token and is the source of the “identity” field in the Z_{base} envelope of every authenticated session [6].

Z_{mcp} (**zap-mcp**). A model context protocol carrying tool calls and structured tool responses. Streaming responses are natural; Z_{mcp} runs as a Z_{ws} stream with structured envelopes.

Z_{acp} (**zap-acp**). An agent control protocol: capability negotiation, session establishment between agents, and structured control messages.

Z_{a2a} (**zap-a2a**). Agent-to-agent task delegation. A task is a structured envelope; we will see that Z_{a2a} embeds in Z_{mcp} by viewing a task as an MCP tool call with structured arguments.

3 The Category $\mathcal{C}_{\text{ZAP}}$

3.1 Protocols as Objects

We work in a labeled transition system model. A protocol P is a tuple $(\Sigma_P, \mathcal{S}_P, s_P^0, \rightarrow_P, \mathcal{V}_P)$ where Σ_P is the message alphabet (set of well-formed envelopes), $\mathcal{S}_P$ is the set of session states, s_P^0 is the initial state, $\rightarrow_P \subseteq \mathcal{S}_P \times \Sigma_P \times \mathcal{S}_P$ is the labelled transition relation, and $\mathcal{V}_P : \mathcal{S}_P \rightarrow 2^{\text{Prop}}$ is a valuation over a fixed propositional vocabulary that includes the security predicates we care about (confidentiality, authenticity, session-bound, post-quantum, fresh-key, channel-bound, etc.).

A *trace* of P is a finite sequence $\sigma = m_1 m_2 \cdots m_n \in \Sigma_P^*$ such that there exist states $s_0, s_1, \dots, s_n$ with $s_0 = s_P^0$ and $s_{i-1} \xrightarrow{m_i}_P s_i$ for all i . Write $\text{Tr}(P)$ for the set of all traces and $\text{Tr}^\omega(P)$ for the set of infinite traces.

3.2 Embeddings as 1-Morphisms

Definition 1 (Protocol embedding). A *protocol embedding* from P to Q , written $\varphi : P \hookrightarrow Q$, is a triple $(\varphi_\Sigma, \varphi_S, \rho)$ such that:

1. $\varphi_\Sigma : \Sigma_P \rightarrow \Sigma_Q^+$ is an injective map that translates each P -message into a non-empty finite sequence of Q -messages.
2. $\varphi_S : \mathcal{S}_P \rightarrow \mathcal{S}_Q$ is a state map with $\varphi_S(s_P^0) = s_Q^0$ when $\rho = \perp$, or $\varphi_S(s_P^0)$ is reachable from s_Q^0 when $\rho \neq \perp$.
3. Whenever $s \xrightarrow{m}_P s'$, the unique trace $\varphi_\Sigma(m) = m_1 m_2 \cdots m_k$ produces transitions $\varphi_S(s) \xrightarrow{m_1}_Q \cdots \xrightarrow{m_k}_Q \varphi_S(s')$.
4. For every security predicate π in the shared vocabulary, $\pi \in \mathcal{V}_P(s) \implies \pi \in \mathcal{V}_Q(\varphi_S(s))$ whenever the embedding declares π as preserved; we write $\Pi(\varphi)$ for the set of such π .
5. ρ is the *embedding's leakage profile*: an upper bound on the additional information that an observer of Q learns about the P -session relative to an observer of P alone. Formally, $\rho \in [0, 1]$ bounds the statistical distance between the distribution of Q -traces under φ and the distribution that would result from running Q on uniformly random input of the same length.

The leakage profile ρ is the categorical placeholder for the “cost of embedding”: for tight embeddings, $\rho = 0$.

Definition 2 (Composition). Given $\varphi : P \hookrightarrow Q$ and $\psi : Q \hookrightarrow R$, the composition $\psi \circ \varphi : P \hookrightarrow R$ is $(\psi_\Sigma^* \circ \varphi_\Sigma, \psi_S \circ \varphi_S, \rho_\varphi + \rho_\psi)$, where ψ_Σ^* extends ψ_Σ to sequences componentwise.

The leakage adds in the standard sub-additive sense (statistical distance is a metric and obeys the triangle inequality); we use addition as a tight upper bound for the small leakages encountered in the family.

Definition 3 (Identity). The identity embedding $\text{id}_P : P \hookrightarrow P$ is $(\text{id}_{\Sigma_P}^{(1)}, \text{id}_{\mathcal{S}_P}, 0)$, where $\text{id}_\Sigma^{(1)}(m) = m$ as a one-element sequence.

3.3 Refinements as 2-Morphisms

Definition 4 (Simulation refinement). Given two embeddings $\varphi, \varphi' : P \hookrightarrow Q$, a *simulation refinement* $\alpha : \varphi \Rightarrow \varphi'$ is a binary relation $R_\alpha \subseteq \mathcal{S}_Q \times \mathcal{S}_Q$ such that:

1. For all $s \in \mathcal{S}_P$, $(\varphi_S(s), \varphi'_S(s)) \in R_\alpha$.
2. Whenever $(t, t') \in R_\alpha$ and $t \xrightarrow{m}_Q t_1$ in the image of φ , there exists a Q -trace $t' \xrightarrow{m'_1}_Q \cdots \xrightarrow{m'_j}_Q t'_j$ in the image of φ' such that $(t_1, t'_j) \in R_\alpha$ and the two paths are observationally equivalent up to the difference in leakage $\rho_{\varphi'} - \rho_\varphi$.

A 2-morphism captures the engineering reality that two embeddings of the same protocol may differ in their wire-level realization without differing in the abstract semantics they realize.

3.4 Category Axioms

Theorem 5 (Category and 2-category). *The data $(\text{Obj}(\mathcal{C}_{\text{ZAP}}), \text{Hom}_{\mathcal{C}_{\text{ZAP}}}, 2\text{Hom}_{\mathcal{C}_{\text{ZAP}}}, \circ, \text{id})$ defines a strict 2-category. In particular, 1-composition is associative and unital, and 2-cells compose horizontally and vertically with the standard interchange law.*

The proof is in the companion document (Theorem 1, `proofs/composability/proof.tex`). The unfolding is straightforward; the only non-trivial check is that the leakage profiles compose by addition across all three axes (vertical, horizontal, identity), which they do because statistical distance is a metric.

4 Composition Theorem

4.1 Tight Embeddings

Definition 6 (Tight embedding). An embedding $\varphi : P \hookrightarrow Q$ is *tight* if its leakage profile satisfies $\rho_\varphi \leq \text{negl}(\lambda)$, where λ is the security parameter and negl denotes any function negligible in λ .

The intuition is that an observer of the outer protocol Q learns nothing more about the inner protocol P than they would learn from a P -observer with access to a uniformly random tape of the same length. The condition rules out side-channel leakage by virtue of the embedding itself; it does not rule out side-channels that come from implementations.

4.2 Inheritance

We model security properties as indistinguishability games. A property π over protocol P is captured by a game G_π^P between a challenger and an adversary $\mathcal{A}$, with advantage $\mathbf{Adv}_\pi^P(\mathcal{A}) := |\Pr[G_\pi^P(\mathcal{A}) = 1] - 1/2|$. The property holds against a class $\mathcal{C}$ of adversaries with parameter λ if $\mathbf{Adv}_\pi^P(\mathcal{A}) \leq \text{negl}(\lambda)$ for all $\mathcal{A} \in \mathcal{C}$.

Theorem 7 (Composition of security under tight embedding). *Let $\varphi : P \hookrightarrow Q$ be a tight embedding and let $\pi \in \Pi(\varphi)$. If P satisfies π against adversary class $\mathcal{C}$ with bound $\epsilon_P(\lambda)$, then Q satisfies π against the same class with bound $\epsilon_Q(\lambda) \leq \epsilon_P(\lambda) + \rho_\varphi + \delta_\varphi(\lambda)$, where δ_φ is a function of the embedding's complexity (see Theorem 3 of the proof companion). For tight embeddings, ρ_φ is negligible and Q inherits π up to a negligible additive term.*

Corollary 8 (Stack inheritance). *If $\varphi_1 : P_1 \hookrightarrow P_2$, $\varphi_2 : P_2 \hookrightarrow P_3$, $\dots$, $\varphi_n : P_n \hookrightarrow P_{n+1}$ are all tight, and P_1 satisfies π with bound ϵ_1 , then P_{n+1} satisfies π with bound bounded by $\epsilon_1 + \sum_{i=1}^n (\rho_{\varphi_i} + \delta_{\varphi_i})$, which is negligible whenever the chain length n is polynomial in λ .*

The proof is by induction on the chain length, with the inductive step being a direct application of Theorem 7.

5 Concrete Embeddings

We exhibit five embeddings used by the family. Each is shown to be tight in the companion proofs document.

5.1 $Z_{\text{http}} \hookrightarrow Z_{\text{ws}}$: HTTP as single-stream WS

Proposition 9. *There is a tight embedding $\varphi_{hw} : Z_{\text{http}} \hookrightarrow Z_{\text{ws}}$ that maps a request/response pair to a singleton bidirectional stream opened, used for one round trip, and closed.*

The embedding takes a Z_{http} request frame to a Z_{ws} **OPEN** frame followed by the same envelope, and a Z_{http} response frame to a Z_{ws} envelope followed by a **CLOSE** frame. The leakage is zero because the only additional bytes are framing bytes that an Z_{http} -only observer would already see at the Z_{base} layer.

5.2 $Z_{\text{fix}} \hookrightarrow Z_{\text{ws}}$: FIX session as persistent stream

Proposition 10. *There is a tight embedding $\varphi_{fw} : Z_{\text{fix}} \hookrightarrow Z_{\text{ws}}$ that maps a FIX session to a single Z_{ws} stream, with FIX sequence numbers preserved as Z_{ws} envelope sequence numbers.*

The map is straightforward: each FIX message is a Z_{ws} envelope, and the FIX session-level reset semantics are realized by Z_{ws} **RESET** frames. The post-quantum properties of the underlying Z_{base} key schedule are inherited; Z_{fix} does not contribute its own key schedule.

5.3 $Z_{\text{rns}} \hookrightarrow Z_{\text{base}}$: RNS into the base envelope

Proposition 11. *There is a tight embedding $\varphi_{rb} : Z_{\text{rns}} \hookrightarrow Z_{\text{base}}$ that injects an RNS-bound identity into the *identity* field of the Z_{base} envelope at handshake time.*

This is the most direct embedding in the family; Z_{rns} contributes exactly one piece of state (the bound identity) and one piece of wire (the channel-binding token), both of which live natively in Z_{base} .

5.4 $Z_{\text{mcp}} \hookrightarrow Z_{\text{ws}}$: MCP over WS

Proposition 12. *There is a tight embedding $\varphi_{mw} : Z_{\text{mcp}} \hookrightarrow Z_{\text{ws}}$ that maps an MCP tool-call/streaming-response pair to a Z_{ws} stream where the request is a single envelope and the response is a finite sequence of envelopes terminated by a final-frame marker.*

5.5 $Z_{\text{a2a}} \hookrightarrow Z_{\text{mcp}}$: A2A as MCP tool calls

Proposition 13. *There is a tight embedding $\varphi_{am} : Z_{\text{a2a}} \hookrightarrow Z_{\text{mcp}}$ that maps an A2A task to an MCP tool call with structured arguments and an MCP streaming response carrying the task progress and result.*

Remark. The embedding $Z_{\text{a2a}} \hookrightarrow Z_{\text{mcp}} \hookrightarrow Z_{\text{ws}} \hookrightarrow Z_{\text{base}}$ makes precise the claim “**zap-a2a** runs over **zap-mcp** runs over **zap-ws** runs over the wire.” The composite embedding is tight by Theorem 8, and so the post-quantum, channel-binding, and authenticity properties of Z_{base} are inherited by Z_{a2a} without a separate proof.

6 Universal Property of Z_{base}

Theorem 14 (Initial object). *Z_{base} is initial in $\mathcal{C}_{\text{ZAP}}$: for every protocol $P \in \text{Obj}(\mathcal{C}_{\text{ZAP}})$, there exists a unique morphism $\iota_P : Z_{\text{base}} \hookrightarrow P$ given by “every Z_{base} envelope is a well-formed P -message in the boot phase before any P -specific state is added.”*

The proof is in the companion document (Theorem 2 of the proof companion). Initiality formalizes a phrase from the family’s design notes: “*The wire is the source of truth.*” Concretely:

1. Any parser that handles Z_{base} frames can recover the structured semantics of any sub-protocol P by composing with the unique ι_P .
2. Any new sub-protocol added to the family must define an embedding into Z_{base} . The category enforces it.
3. Wire-level interoperability across language bindings is a functor: each binding L is a category $\mathcal{C}_L$ of “protocol-as-implemented-in- L ,” and there is a functor $F_L : \mathcal{C}_{\text{ZAP}} \rightarrow \mathcal{C}_L$ that takes Z_{base} to its in-language envelope type. By initiality, every sub-protocol’s binding type is determined up to isomorphism by the binding for Z_{base} .

7 Practical Implications

7.1 Security review reduces to embedding review

For a new sub-protocol P_{new} , the security review need not re-prove handshake security, post-quantum properties, or channel-binding. It suffices to:

1. Define the embedding $\iota_{P_{\text{new}}} : Z_{\text{base}} \hookrightarrow P_{\text{new}}$ (always exists by initiality).
2. Show the embedding is tight.
3. Identify the security properties π that Z_{base} already provides and that the embedding preserves ($\pi \in \Pi(\iota_{P_{\text{new}}})$).
4. Prove only the P_{new} -specific properties on top.

This is a substantial reduction in proof burden. For example, when Z_{mcp} was added to the family, the security review consisted of showing that the MCP-over-WS embedding is tight (a one-page proof, Theorem 12) and that MCP’s tool-call authorization adds a single new property on top of the inherited Z_{base} guarantees. No re-proof of the post-quantum hybrid KEM was needed.

7.2 Wire compatibility

A Z_{base} parser is universal in the family. Tooling (Wireshark dissectors [7], language bindings, capture analyzers) need to be written once at the Z_{base} layer; the structured semantics of any sub-protocol are recovered by composition with the relevant embedding.

7.3 Implementation languages as functors

The 30+ language bindings under `zap-go`, `zap-rust`, `zap-java`, and so on are each a functor $F_L : \mathcal{C}_{\text{ZAP}} \rightarrow \mathcal{C}_L$. The categorical structure is preserved: an embedding $\varphi : P \hookrightarrow Q$ in $\mathcal{C}_{\text{ZAP}}$ becomes an embedding $F_L(\varphi)$ in $\mathcal{C}_L$. This means cross-language interoperability is automatic at the categorical level: an A2A agent written in Rust calling an MCP server written in Go is a diagram in $\mathcal{C}_{\text{wire}}$ that commutes by functoriality.

7.4 Stack security in one example

Example 15 (Inherited PQ properties for FIX-over-RNS). Consider an institutional client running Z_{fix} (FIX-compatible session) over Z_{rns} (identity binding) over Z_{ws} over Z_{base} . The chain of embeddings is $Z_{\text{fix}} \hookrightarrow Z_{\text{ws}} \hookrightarrow Z_{\text{base}} \leftarrow Z_{\text{rns}} \hookrightarrow Z_{\text{base}}$, with the Z_{rns} contribution an additive binding rather than a layering. By Theorem 8 and the $Z_{\text{rns}} \hookrightarrow Z_{\text{base}}$ tightness, the post-quantum hybrid-KEM key schedule of Z_{base} is inherited by Z_{fix} , and the identity-binding guarantee of Z_{rns} holds at every Z_{fix} message. No FIX-specific post-quantum proof is needed.

7.5 Where the framework does not help

The framework gives no help against side-channel attacks below the LTS abstraction (timing, cache, electromagnetic), against implementation bugs that violate the tightness condition (e.g., an implementation that leaks session state via debug logs), or against adversaries outside the modeled class (e.g., adversaries with access to the host’s memory). It also says nothing about *availability* composition; a denial-of-service attack on Z_{base} denies service to every sub-protocol.

8 Related Work

Universal Composability. Canetti’s UC framework [1, 8] is the gold standard for composable cryptographic security. Our framework is strictly weaker: we model protocols as labelled transition systems and security properties as indistinguishability games, without the UC machinery of environments, ideal functionalities, and the composition theorem. The trade is deliberate: UC proofs are expensive, and the ZAP family does not need the full strength of UC because the embeddings are tight by construction. For sub-protocols where UC-strength composition is required (e.g., Z_{rns} when used as a stand-alone identity provider), a UC proof is given separately and the categorical framework is used to lift it to the rest of the family.

Multiparty session types. Honda, Yoshida, and Carbone [2] give a type system in which multiparty protocols are specified globally and projected to local types per participant. Our category $\mathcal{C}_{\text{ZAP}}$ can be viewed as a coarser, “protocol-level” analogue of session types: objects are protocols rather than session types per se, and morphisms are embeddings rather than projections. We do not pursue the projection direction here; that would be the natural follow-up.

Reactive simulatability. Pfitzmann and Waidner [3, 9] develop a model close in spirit to UC but with an explicit notion of reactive systems and a composition theorem in that setting. Our 2-cells (simulation refinements) are inspired by their notion of “perfect indistinguishability of structures.”

Process algebra. CSP [10] and the π -calculus [11] provide the LTS substrate we work in. Our embeddings are a categorical variant of CSP-style refinement under hiding, with the security predicates added on top.

Categorical accounts of protocols. Caires and Pfenning’s session types as intuitionistic linear propositions [12] and Wadler’s “Propositions as Sessions” [13] give a categorical foundation for session-typed concurrency. We work at a coarser level (protocol families rather than individual sessions); the two views are complementary.

9 Conclusion

The ZAP family forms a strict 2-category $\mathcal{C}_{\text{ZAP}}$. The base wire format Z_{base} is initial. Tight embeddings preserve security properties under composition with explicit and tight bounds. The seven sub-protocols embed into one another along the diagram of Section 5, and the practical engineering claim that any composite stack inherits the security of its components is, in the framework, a calculation. Future work includes (i) a Coq formalization of $\mathcal{C}_{\text{ZAP}}$ with the embeddings of Section 5 mechanically verified, (ii) the projection from $\mathcal{C}_{\text{ZAP}}$ to a multiparty session-type calculus [2], and (iii) a denotational semantics in which the leakage profile ρ is given by an information-theoretic mutual-information bound rather than by statistical distance.

References

- [1] Ran Canetti. “Universally Composable Security”. In: *Journal of the ACM* 67.5 (2020), 28:1–28:94.
- [2] Kohei Honda, Nobuko Yoshida, and Marco Carbone. “Multiparty Asynchronous Session Types”. In: *Journal of the ACM* 63.1 (2016), 9:1–9:67.
- [3] Birgit Pfitzmann and Michael Waidner. “A Model for Asynchronous Reactive Systems and its Application to Secure Message Transmission”. In: *IEEE Symposium on Security and Privacy*. 2001, pp. 184–200.
- [4] Karthikeyan Bhargavan et al. “Triple Handshakes and Cookie Cutters: Breaking and Fixing Authentication over TLS”. In: *IEEE Symposium on Security and Privacy*. 2014, pp. 98–113.
- [5] ZAP Protocol Authors. *Post-Quantum Hybrid KEM for ZAP Sessions*. <https://zap-protocol.io>. 2026.
- [6] ZAP Protocol Authors. *RNS: Identity Binding for the ZAP Protocol Family*. <https://zap-protocol.io>. 2026.
- [7] ZAP Protocol Authors. *The ZAP Wire Format Specification*. <https://zap-protocol.io>. 2026.
- [8] Ran Canetti. “Universally Composable Security: A New Paradigm for Cryptographic Protocols”. In: *42nd Annual Symposium on Foundations of Computer Science (FOCS)*. 2001, pp. 136–145.
- [9] Birgit Pfitzmann and Michael Waidner. “Composition and Integrity Preservation of Secure Reactive Systems”. In: *7th ACM Conference on Computer and Communications Security (CCS)*. 2000, pp. 245–254.
- [10] C. A. R. Hoare. *Communicating Sequential Processes*. Prentice-Hall, 1985.
- [11] Robin Milner. *Communicating and Mobile Systems: The π -Calculus*. Cambridge University Press, 1999.
- [12] Luís Caires and Frank Pfenning. “Session Types as Intuitionistic Linear Propositions”. In: *CONCUR* (2010), pp. 222–236.
- [13] Philip Wadler. “Propositions as Sessions”. In: *ICFP*. 2012, pp. 273–286.